QUICKSTOCK

José Ítalo S. C. Dantas
Leticia Viviane Pereira da Silva
Luan Feitosa Santos
Marcelo Vitor Viana da Silva

INTEGRAÇÃO DO APLICATIVO MÓVEL QUICKSTOCK COM A API DE BACK-END

Caruaru
2026

QUICKSTOCK

José Ítalo S. C. Dantas
Leticia Viviane Pereira da Silva
Luan Feitosa Santos
Marcelo Vitor Viana da Silva

INTEGRAÇÃO DO APLICATIVO MÓVEL QUICKSTOCK COM A API DE BACK-END

Relatório técnico que registra a API já em funcionamento e as alterações feitas para ligar o aplicativo móvel ao back-end, em especial o perfil de cadastro, a recuperação de senha e o módulo de logística. O cenário de demonstração do sistema usa empresas de Caruaru e região.

Caruaru
2026

RESUMO

O QuickStock é um sistema de estoque, compras e operação de quiosques. O aplicativo, escrito em React Native com Expo, conversa com uma API REST em Java 17, Spring Boot 3.4 e PostgreSQL. Antes da integração descrita aqui, produtos, quiosques, marketplace, endereços, pagamentos, notificações e o resumo financeiro já eram lidos e gravados por essa API. O cadastro não persistia a escolha entre cliente, fornecedor ou os dois papéis: a conta nascia com perfil administrativo e a empresa ficava como compradora, enquanto o aplicativo guardava a escolha apenas no aparelho. A tela de nova senha chamava uma rota que não existia. Caminhoneiros e logística usavam listas fixas na tela. Este relatório explica o que já estava pronto e o que foi acrescentado: a coluna de perfil de uso, a classificação da empresa, a rota de recuperação de senha, a entrada de empresas dos dois papéis no marketplace e os recursos de caminhoneiro e de resumo logístico. O atalho de login que não passava pela API foi removido do aplicativo.

Palavras-chave: QuickStock. API REST. Aplicativo móvel. Cadastro. Logística.

LISTA DE ILUSTRAÇÕES

Figura 1 – Camadas da API e o ponto em que o aplicativo entra 9
Figura 2 – Perfil escolhido no cadastro e tipo gravado na empresa 13
Figura 3 – Caminho do cadastro até as abas do aplicativo 14
Figura 4 – Empresa, caminhoneiro e entrega 15
Figura 5 – Etapas montadas para a tela de logística 16

LISTA DE TABELAS

Tabela 1 – Recursos que o aplicativo já consumia 11
Tabela 2 – Perfil de uso e tipo da empresa 12
Tabela 3 – Rotas acrescentadas nesta integração 16
Tabela 4 – Status da entrega e texto mostrado na tela 16

LISTA DE SIGLAS

API Interface de programação de aplicações

CNPJ Cadastro Nacional da Pessoa Jurídica

CPF Cadastro de Pessoas Físicas

HTTP Protocolo de transferência de hipertexto

JSON Notação de objeto JavaScript

JWT JSON Web Token

REST Transferência de estado representacional

SQL Linguagem de consulta estruturada

SUMÁRIO

RESUMO 2
LISTA DE ILUSTRAÇÕES 3
LISTA DE TABELAS 4
LISTA DE SIGLAS 5
1 INTRODUÇÃO 7
1.1 Objetivo 7
1.2 Delimitação 7
2 O SISTEMA QUICKSTOCK 7
2.1 O aplicativo 7
2.2 API 8
2.3 A ligação entre os dois 9
3 O QUE JÁ ESTAVA PRONTO 9
3.1 Organização do código 9
3.2 Dados já persistidos 10
3.3 Recursos já expostos 10
3.4 Autenticação anterior 11
3.5 Onde a ligação ainda falhava 11
4 O QUE FOI ALTERADO 12
4.1 Perfil de uso no cadastro e no login 12
4.2 Efeito no marketplace 14
4.3 Recuperação de senha 14
4.4 Caminhoneiros e entregas 15
4.5 Resumo mostrado na tela de logística 15
4.6 Ajustes correspondentes no aplicativo 16
5 LIMITES DESTA ENTREGA 17
6 CONSIDERAÇÕES FINAIS 17
REFERÊNCIAS 17

1 INTRODUÇÃO

Operar estoque, compra e quiosque no QuickStock depende de duas partes que precisam concordar sobre os mesmos dados. De um lado está o aplicativo móvel. Do outro, a API que grava usuários, empresas, produtos, pedidos e pagamentos no PostgreSQL. A maior parte dessa conversa já existia. O que faltava era fazer o cadastro, a sessão e a logística obedecerem ao que as telas novas passaram a pedir.

Este relatório foi escrito a partir do código dos dois projetos, no estado de setembro de 2026. Ele separa, de propósito, o que já funcionava do que foi construído na integração. A separação evita tratar rota antiga como novidade e evita descrever como pronto um comportamento que a tela apenas simulava.

1.1 Objetivo

O objetivo é registrar, em linguagem de quem vai manter o sistema, três resultados. O primeiro é o mapa da API que o aplicativo já usava. O segundo é a persistência do perfil escolhido na abertura da conta, junto com a troca de senha pedida pela tela correspondente. O terceiro é a substituição das listas fixas de caminhoneiro e de logística por recursos HTTP próprios, sempre ligados à empresa da pessoa logada.

1.2 Delimitação

O texto cobre a API em Spring Boot e os pontos do aplicativo que precisaram mudar para consumi-la. Telas de produto, quiosque, sacola, endereço e pagamento já chamavam a API e não foram reescritas. Também não entra aqui o desenho visual dessas telas. O rastreio descrito na logística é o que a API calcula a partir do status gravado. Não há integração com operadora de transporte, mapa ou mensagem de e-mail.

2 O SISTEMA QUICKSTOCK

O QuickStock atende quem vende em quiosque e quem compra de distribuidoras. O mesmo aplicativo muda de conjunto de abas conforme o papel da conta. Cliente percorre exploração, carrinho e pedidos. Fornecedor, e também quem acumula os dois papéis, trabalha com estoque, quiosque, compras e o painel da empresa. Essa escolha de interface só é confiável se o papel vier do servidor, e não de um valor esquecido no aparelho.

2.1 O aplicativo

O aplicativo está em React Native com Expo e TypeScript. A navegação separa o fluxo de entrada (animação, boas-vindas, escolha de papel, cadastro, login e senha esquecida) do

fluxo autenticado. Serviços em `src/services` concentram as chamadas HTTP. O endereço base é a porta 8080. No emulador Android o host de desenvolvimento é 10.0.2.2. No aparelho, o host sai do endereço que o Expo informa. No restante dos casos, usa-se localhost.

A sessão guarda o token, o usuário e o instante de expiração. Ao reabrir o aplicativo, a rota `GET /api/usuarios/me` confirma o token e devolve o usuário atual. Se a resposta falha, a sessão local é descartada.

2.2 API

A API usa Java 17 e Spring Boot 3.4. A persistência é Spring Data JPA com Hibernate, dialeto PostgreSQL, e o esquema é atualizado na subida (`ddl-auto=update`). O script `data.sql` roda em seguida e aplica ajustes idempotentes: colunas que ainda não existem, cargas de demonstração e correções que podem ser repetidas sem duplicar linha. A senha é armazenada com BCrypt. O token de sessão é um JWT (JSON Web Token), com validade de 24 horas, definida em `jwt.expiration-ms`. A documentação interativa fica em `/swagger`.

O código segue camadas. O controller recebe HTTP e devolve JSON. O service concentra a regra. O repository fala com o banco. Entidade é o que se persiste. DTO é o que atravessa a fronteira HTTP, para não expor o hash da senha nem detalhes internos da tabela. A Figura 1 mostra essa divisão e o lugar do aplicativo.

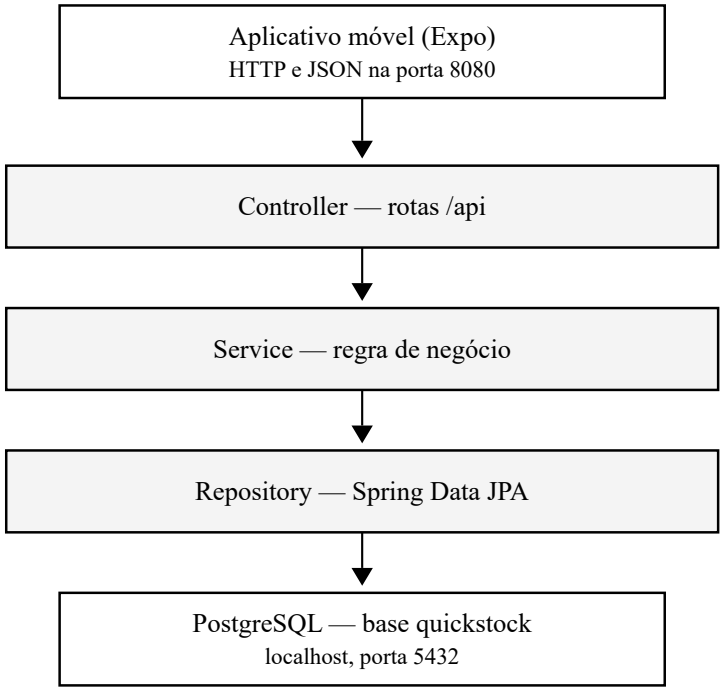


Figura 1 – Camadas da API e o ponto em que o aplicativo entra
Fonte: Elaborada pelos autores (2026).

2.3 A ligação entre os dois

Cada tela que precisa de dado remoto chama um serviço. O serviço monta a URL, envia JSON quando há corpo e lê o campo `erro` quando a resposta não é de sucesso. Não há um gateway separado: o aplicativo fala direto com a API. Imagem de produto gravada em disco é devolvida como caminho e o aplicativo prefixa o mesmo host da API.

Dois conceitos de "perfil" convivem e não devem ser misturados. O perfil administrativo (`perfis`, hoje o registro Admin) é o papel interno da conta dentro da empresa e continua sendo atribuído no cadastro público. O perfil de uso é outra informação: cliente, fornecedor ou os dois. É ele que decide as abas. Antes da integração, só o primeiro era gravado no banco.

3 O QUE JÁ ESTAVA PRONTO

3.1 Organização do código

O pacote `com.quickstock.backend` já reunia config, controller, dto, entity, exception, repository e service. Erros de validação e de regra de cadastro saem num JSON com a chave `erro`, no formato que o aplicativo já sabe mostrar. Os controllers declaram CORS aberto, o

que permite o Expo Web chamar a API a partir de outra origem. No aplicativo nativo essa liberação não é exigida pelo aparelho, mas não atrapalha.

A subida da aplicação cria ou ajusta tabelas e, depois, executa `data.sql`. Esse script já inseria o perfil Admin, colunas de estoque e de pedido, cartões salvos e três distribuidoras de demonstração: Casa dos Vinhos, Cervejaria Caruaru e Whisky Labs, com produtos e imagens.

3.2 Dados já persistidos

As entidades anteriores à logística cobrem o núcleo operacional. Empresa guarda nome, CNPJ, telefone, tipo, descrição e imagens. Usuário guarda nome, e-mail único, hash da senha, perfil administrativo, empresa e a flag de ativo. Produto pertence a uma empresa e tem preço, unidade, estoque, código e imagem. Barraca (o quiosque na interface) liga um evento a um responsável e a itens de estoque. Pedido, item de pedido e pagamento registram a venda. Solicitação de compra, com seus itens, é o pedido feito a uma distribuidora. Endereço de entrega, forma de pagamento salva e cartão salvo (sem o código de segurança) completam o checkout. Evento organiza as barracas.

O campo `tipo` da empresa já distinguia, entre outros valores, COMPRADOR e DISTRIBUIDOR. O marketplace listava somente DISTRIBUIDOR. Uma empresa cadastrada pela tela pública nascia com o valor padrão COMPRADOR e, por isso, não aparecia como fornecedora, mesmo quando a pessoa tinha se declarado fornecedora no aplicativo.

3.3 Recursos já expostos

A Tabela 1 reúne os recursos que as telas já chamavam antes desta integração. A lista não é o catálogo inteiro da API: há também eventos, pagamentos avulsos, itens de pedido e perfis administrativos, usados pela própria API e por cargas internas. O recorte abaixo é o contrato que o aplicativo já dependia.

Tabela 1 – Recursos que o aplicativo já consumia

Recurso	Uso no aplicativo
`POST /api/cadastro`	Abertura de conta, com empresa e usuário no mesmo corpo.
`POST /api/usuarios/login`	E-mail, senha e devolução do token com o usuário.
`GET /api/usuarios/me`	Reidratação da sessão a partir do token.
`PUT /api/usuarios/{id}`	Nome, e-mail e senha nas configurações.
`PUT /api/empresas/{id}`	Nome, CNPJ e telefone da empresa.
`/api/produtos`	Listagem, detalhe, inclusão, edição, exclusão e upload de imagem.
`/api/barracas`	Quiosques da empresa, estoque e desativação.
`/api/marketplace`	Fornecedores e o catálogo de cada um.
`/api/solicitacoes-compra`	Pedido de compra e consulta do andamento.
`/api/enderecos`	Endereços da empresa e consulta de CEP.
`/api/formas-pagamento`	PIX, crédito, débito e dinheiro salvos na empresa.
`/api/cartoes-pagamento`	Cartões de crédito e débito, sem armazenar o CVV.
`/api/financeiro`	Resumo e movimento do dia no painel.
`/api/notificacoes`	Avisos da empresa compradora.

Fonte: Elaborada pelos autores (2026), a partir dos serviços do aplicativo e dos controllers da API.

3.4 Autenticação anterior

O login comparava o e-mail informado com o registro, exigia usuário ativo e conferia a senha com BCrypt. Em caso de acerto, a API emitia o JWT e devolvia também o usuário, já no DTO, sem o hash. O cadastro público criava a empresa, localizava o perfil Admin (ou Gestor, ou Gerente, se o nome Admin não existisse) e gravava o usuário ligado aos dois. E-mail repetido respondia 409. CNPJ repetido também.

Havia ainda, só no aplicativo, um desvio: e-mail `admin` e senha `1234` montavam uma sessão falsa, com token que a API não reconhece e com uma empresa inventada no cliente. Esse caminho não lia nem gravava nada no PostgreSQL. Quem testava por ele via abas de cliente e fornecedor ao mesmo tempo, sem que essa combinação existisse no banco.

3.5 Onde a ligação ainda falhava

Três furos impediam tratar o conjunto como um sistema só. O primeiro era o perfil de uso. A tela anterior ao formulário pergunta se a pessoa é cliente, fornecedora ou as duas coisas. O formulário mudava os campos (CPF e telefone para cliente; empresa e CNPJ para fornecedor; os dois blocos quando a pessoa acumula os papéis). Na hora de enviar, porém, o corpo não levava essa escolha. A API ignoraria o campo se ele viesse, porque o DTO de cadastro não o declarava. O aplicativo então salvava a escolha no armazenamento local, indexada pelo e-mail. Trocar de aparelho, limpar os dados ou entrar numa instalação nova

fazia a conta cair na interface de empresa, que é o ramo usado quando o perfil local não é "Cliente".

O segundo furo era a senha. A tela "esqueceu a senha" envia e-mail e a nova senha para `POST /api/usuarios/recuperar-senha`. Essa rota não existia. A chamada terminava em erro e a senha no banco permanecia a antiga.

O terceiro furo era a logística. As telas de caminhoneiros, de cadastro e de envio estavam na navegação e no atalho da home da empresa, mas os nomes, rotas e a linha do tempo ("em trânsito", pedido 1049, motorista Rafael) eram constantes no código da tela. Cadastrar um motorista só alterava o estado daquela visita. Ao sair e voltar, a lista fixa reaparecia.

4 O QUE FOI ALTERADO

A integração não substituiu a API. Ela fechou os três furos da seção anterior e fez o marketplace enxergar fornecedor cadastrado pela própria tela. Contas antigas não foram reclassificadas: quem já estava como COMPRADOR continua comprador. O script apenas preenche `perfil_uso` com Fornecedor quando a coluna está vazia, para a interface não ficar sem papel.

4.1 Perfil de uso no cadastro e no login

A tabela `usuarios` ganhou `perfil_uso` (Cliente, Fornecedor ou ClienteFornecedor) e `cpf`, opcional, com até 14 caracteres. O `data.sql` cria as colunas se elas ainda não existirem e preenche o perfil das linhas antigas. O cadastro passou a exigir, no corpo, o campo `perfil`. Valor ausente é tratado como Fornecedor, para não quebrar um cliente antigo da API. Valor fora da lista responde 400, com a mensagem de perfil inválido.

O mesmo cadastro define `empresa.tipo` a partir do perfil, conforme a Tabela 2. O perfil administrativo da linha de usuário continua sendo Admin. São colunas diferentes, com funções diferentes. O CPF informado no usuário é guardado só com dígitos. Se o documento da empresa já existir, a mensagem fala em CPF quando há 11 dígitos e em CNPJ nos demais casos. Isso cobre o cliente, que envia o CPF no lugar do CNPJ da empresa, porque a conta ainda precisa de uma empresa para o restante da API, que é toda organizada por `empresaId`.

Tabela 2 – Perfil de uso e tipo da empresa

Escolha na tela	perfil_uso	empresa.tipo	Aparece como fornecedor
Sou cliente	Cliente	CLIENTE	Não
Sou fornecedor	Fornecedor	DISTRIBUIDOR	Sim
Cliente e fornecedor	ClienteFornecedor	AMBOS	Sim

Fonte: Elaborada pelos autores (2026).

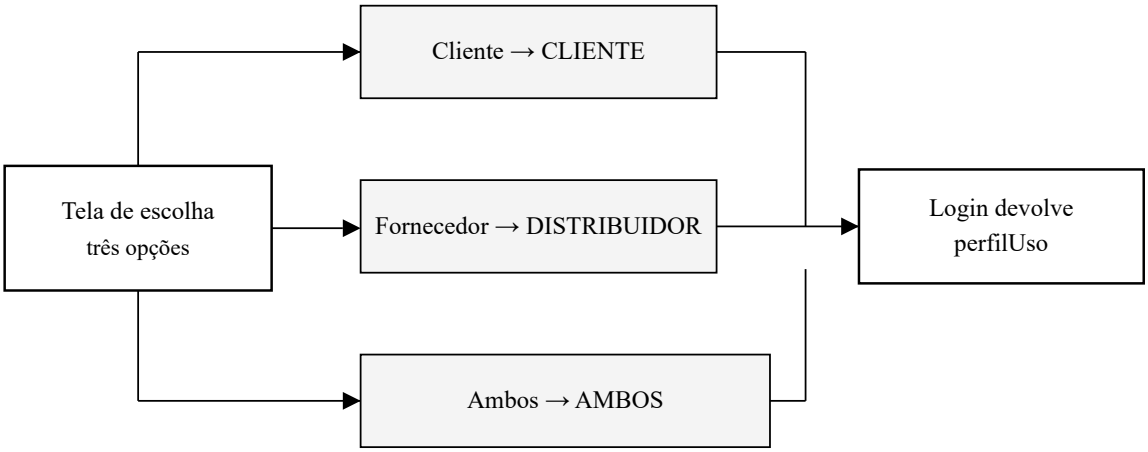


Figura 2 – Perfil escolhido no cadastro e tipo gravado na empresa
Fonte: Elaborada pelos autores (2026).

A resposta de usuário, usada no cadastro, no login e em `/me`, passou a incluir `perfilUso`. Se a coluna estiver vazia numa linha antiga, a resposta assume Fornecedor. Login e recuperação de senha localizam o e-mail sem diferenciar maiúsculas de minúsculas. A rota `/me` continua achando a pessoa pelo identificador que está dentro do token; o que mudou nela é o conteúdo da resposta, não a forma de busca.

No aplicativo, o cadastro envia `perfil` junto com empresa e usuário. Depois do login, o contexto de autenticação lê `perfilUso` do usuário devolvido. O armazenamento local permanece apenas como reserva, para uma resposta antiga que ainda não traga o campo. O desvio `admin` / `1234` foi retirado: entrar no sistema exige uma conta real.

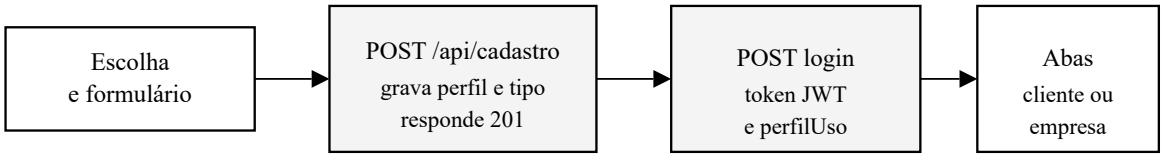


Figura 3 – Caminho do cadastro até as abas do aplicativo
Fonte: Elaborada pelos autores (2026).

4.2 Efeito no marketplace

Listar fornecedores, abrir o catálogo de um fornecedor, aceitar uma solicitação de compra e montar notificações de destaque usavam a lista fixa que continha só DISTRIBUIDOR. Com a Tabela 2, a pessoa que se cadastra como fornecedora já nasce nesse tipo e passa a aparecer. Quem marca os dois papéis nasce como AMBOS. Esses três pontos da API passaram a aceitar DISTRIBUIDOR e AMBOS. CLIENTE fica de fora da vitrine, porque essa conta compra e não oferece catálogo de distribuidora.

As cargas de demonstração que procuram as distribuidoras parceiras continuam filtrando DISTRIBUIDOR. Elas não foram alargadas para AMBOS, para não misturar conta recém-criada com o catálogo de vitrine que o script semeia na Casa dos Vinhos, na Cervejaria Caruaru e na Whisky Labs.

4.3 Recuperação de senha

`POST /api/usuarios/recuperar-senha` recebe `email` e `novaSenha`. E-mail em branco responde 400. Senha com menos de seis caracteres responde 400. E-mail que não existe responde 404, com `erro` igual a “E-mail não encontrado.”. Quando o e-mail existe, a API gera um hash BCrypt novo, grava em `senha_hash` e responde 200 com a mensagem de senha atualizada. A tela que já existia no aplicativo não precisou de contrato novo: ela já enviava esse JSON.

A troca é imediata. Não há código enviado por e-mail nem prazo de validade do link, porque a tela pede a nova senha no próprio formulário. Quem conhece um e-mail cadastrado e alcança a API consegue definir outra senha. Esse limite está registrado na seção 5, para não ser lido como confirmação de identidade.

4.4 Caminhoneiros e entregas

Duas tabelas novas guardam a operação de entrega da empresa logada. `caminhoneiros` tem empresa, nome, veículo, rota, status e data de criação. O status previsto na listagem é Em rota, Carregando, Disponível ou Atrasado. `entregas_logistica` pertence a um caminhoneiro e tem nome do cliente, cidade, rota, status e data de criação. O status da entrega é Em rota, Carregando ou Entregue.

`GET /api/caminhoneiros?empresaId=` devolve a lista da empresa, da mais recente para a mais antiga, cada item com as entregas e com a lista de nomes de cliente usada no cartão da tela. Sem `empresaId`, a resposta é uma lista vazia. `POST /api/caminhoneiros` exige empresa, nome, veículo e rota. A empresa inexistente responde 400. Se o corpo traz entrega com nome de cliente, o caminhoneiro nasce Carregando e a entrega também, com a rota do motorista. Cidade em branco vira “Local não informado”. Sem nome de cliente, o caminhoneiro nasce Disponível e sem entregas.



Figura 4 – Empresa, caminhoneiro e entrega
Fonte: Elaborada pelos autores (2026).

4.5 Resumo mostrado na tela de logística

`GET /api/logistica?empresaId=` escolhe a entrega que a tela deve contar. Primeiro busca entregas da empresa cujo status não é Entregue e fica com a de maior identificador. Se não houver entrega em aberto, usa a entrega mais recente, inclusive a já concluída. Se não houver nenhuma, a resposta vem com `vazio` verdadeiro e a lista de etapas vazia. A tela, nesse caso, mostra “Sem entregas” em vez da linha do tempo fixa que existia antes. Empresa inexistente responde 400.

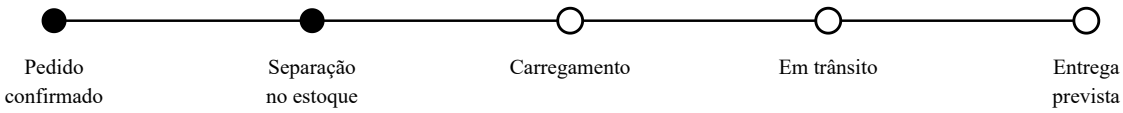
Quando há entrega, a API monta cinco etapas com horário a partir de `criado_em`: pedido confirmado no instante da gravação, separação 30 minutos depois, carregamento aos 65 minutos, trânsito aos 135 e entrega prevista aos 230. Quantas delas aparecem concluídas depende do status, como na Tabela 4. O número mostrado no cartão “Pedido” é o identificador da entrega, no formato `#` seguido do id. Não é o número de uma solicitação de

compra do marketplace. O tempo “2h 15min” é o texto fixo do estado em trânsito, não um cálculo de rota.

Tabela 4 – Status da entrega e texto mostrado na tela

Status da entrega	Etapas concluídas	Texto do status	Tempo estimado
Carregando (e os demais não listados)	As duas primeiras	Em separação	Aguardando carregamento
Em rota	As quatro primeiras	Em trânsito	2h 15min
Em rota, com caminhoneiro Atrasado	As quatro primeiras	Atrasado	Atraso na rota
Entregue	As cinco	Entregue	Concluída

Fonte: Elaborada pelos autores (2026), a partir de LogisticaService.



Exemplo com entrega em Carregando: só os dois primeiros marcos estão concluídos.

Figura 5 – Etapas montadas para a tela de logística
Fonte: Elaborada pelos autores (2026).

O cadastro feito pela tela cria a entrega em Carregando. Os status Em rota, Entregue e Atrasado existem no modelo e na leitura, para a linha do tempo e para o selo da lista. A tela de cadastro ainda não oferece um controle para avançar esses status. Quem for operar o andamento completo precisará de uma atualização posterior, ou de gravar o status por outro cliente da API. Isso também está entre os limites da seção 5.

4.6 Ajustes correspondentes no aplicativo

Um serviço novo, `logisticaService.ts`, lista caminhoneiros, cria o registro e busca o resumo. As três telas deixaram de usar vetores constantes. Elas leem `user.empresa.id` da sessão, recarregam ao ganhar foco e tratam lista vazia, falha de rede e ausência de empresa. O contador de motoristas ativos passou a ser o tamanho da lista devolvida, não o número 18 que estava escrito na tela. A Tabela 3 fecha o contrato acrescentado.

Tabela 3 – Rotas acrescentadas nesta integração

Rota	Função
`POST /api/usuarios/recuperar-senha`	Troca o hash da senha do e-mail informado.
`GET /api/caminhoneiros?empresaId=`	Lista motoristas e entregas da empresa.
`POST /api/caminhoneiros`	Cria motorista e, se houver cliente, a primeira entrega.
`GET /api/logistica?empresaId=`	Resume a entrega ativa, ou informa que não há entrega.

Fonte: Elaborada pelos autores (2026).

O cadastro público existente, `POST /api/cadastro`, não mudou de endereço. Mudou o corpo aceito e o efeito colateral na empresa. O login também manteve o endereço e passou a devolver `perfilUso` dentro de `usuario`.

5 LIMITES DESTA ENTREGA

Conta criada antes da coluna `perfil_uso` recebe Fornecedor na migração e mantém o tipo de empresa que já tinha. Se esse tipo for COMPRADOR, a pessoa vê a interface de empresa, mas não entra na vitrine como distribuidora. Só o cadastro novo aplica a Tabela 2.

A recuperação de senha não prova que quem envia o formulário controla a caixa de e-mail. Ela atende o desenho atual da tela, que coleta a nova senha na hora. Um fluxo com mensagem e token de uso único seria outra entrega.

A linha do tempo não observa GPS nem pedido do marketplace. Os horários são deslocamentos fixos sobre a data de criação da entrega, e o "pedido" exibido é o id dessa entrega. O status Atrasado só altera o texto quando a entrega já está Em rota; o formulário do aplicativo não grava essa transição.

As demais telas continuam como estavam. Produto, quiosque, sacola, endereço, cartão e financeiro não foram reabertos nesta integração, porque já tinham rota correspondente.

6 CONSIDERAÇÕES FINAIS

O aplicativo e a API já formavam um sistema na maior parte do fluxo comercial. A falha estava no que a interface decidia sozinha: o papel da conta, a senha esquecida e a logística de demonstração. Com o perfil gravado no usuário, o tipo da empresa alinhado a esse perfil e as três rotas novas, essas telas passam a ler e gravar o mesmo PostgreSQL que o restante do QuickStock.

Para ver as rotas novas, a API precisa ser reiniciada depois da atualização do código, de modo que o Hibernate crie `caminhoneiros` e `entregas_logistica` e o script preencha `perfil_uso` nas contas já existentes. O login de teste que não consultava o banco não faz mais parte do aplicativo.

REFERÊNCIAS

EXPO. **Expo documentation**. Disponível em: https://docs.expo.dev/. Acesso em: 27 set. 2026.

FIELDING, R. T. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Ciência da Informação e Computação) – University of California, Irvine, 2000. O capítulo 5 define o estilo REST usado pela API do QuickStock.

JONES, M.; BRADLEY, J.; SAKIMURA, N. **RFC 7519**: JSON Web Token (JWT). Fremont: IETF, 2015. Disponível em: https://www.rfc-editor.org/rfc/rfc7519. Acesso em: 27 set. 2026.

ORACLE. **Java SE 17 documentation**. Disponível em: https://docs.oracle.com/en/java/javase/17/. Acesso em: 27 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation**. Disponível em: https://www.postgresql.org/docs/. Acesso em: 27 set. 2026.

SPRING. **Spring Boot 3.4 reference documentation**. Disponível em: https://docs.spring.io/spring-boot/docs/3.4.0/reference/html/. Acesso em: 27 set. 2026.

SPRING. **Spring Data JPA reference documentation**. Disponível em: https://docs.spring.io/spring-data/jpa/reference/. Acesso em: 27 set. 2026.